



Node.js

Introdução ao ambiente de execução JavaScript no servidor — da instalação ao desenvolvimento de APIs completas com Express.

Professor Flávio Mota

O que é o Node.js?

Back end



Executa JavaScript diretamente no servidor, permitindo criar APIs, servidores web e aplicações de alta escalabilidade com o mesmo código já conhecido do front end.

Front end



Originalmente, o JavaScript era utilizado apenas no navegador, manipulando HTML, CSS e interações do usuário. O Node.js expandiu essa linguagem para o lado do servidor.

Banco de dados



Node.js é um interpretador de JavaScript assíncrono com código aberto orientado a eventos, focado em migrar a programação JavaScript do cliente (*frontend*) para os servidores, criando aplicações de alta escalabilidade — como um servidor web — capaz de manipular milhares de conexões simultâneas em tempo real.

Características do Node.js

O Node.js é um ambiente JavaScript desenvolvido para ser executado em servidores e possui características que o tornam altamente eficiente e produtivo:

Motor V8 da Google

Um motor de execução otimizado e extremamente rápido, o mesmo que roda o Chrome, compilando JavaScript diretamente para código de máquina.

Assíncrono por padrão

Código executado de forma assíncrona — nada é bloqueante. Isso garante que o servidor continue respondendo enquanto aguarda operações de I/O.

Baseado em eventos

Projeto baseado em iteração de eventos de forma parecida com os navegadores modernos, tornando o modelo de programação familiar para desenvolvedores web.

Acesso à rede como prioridade

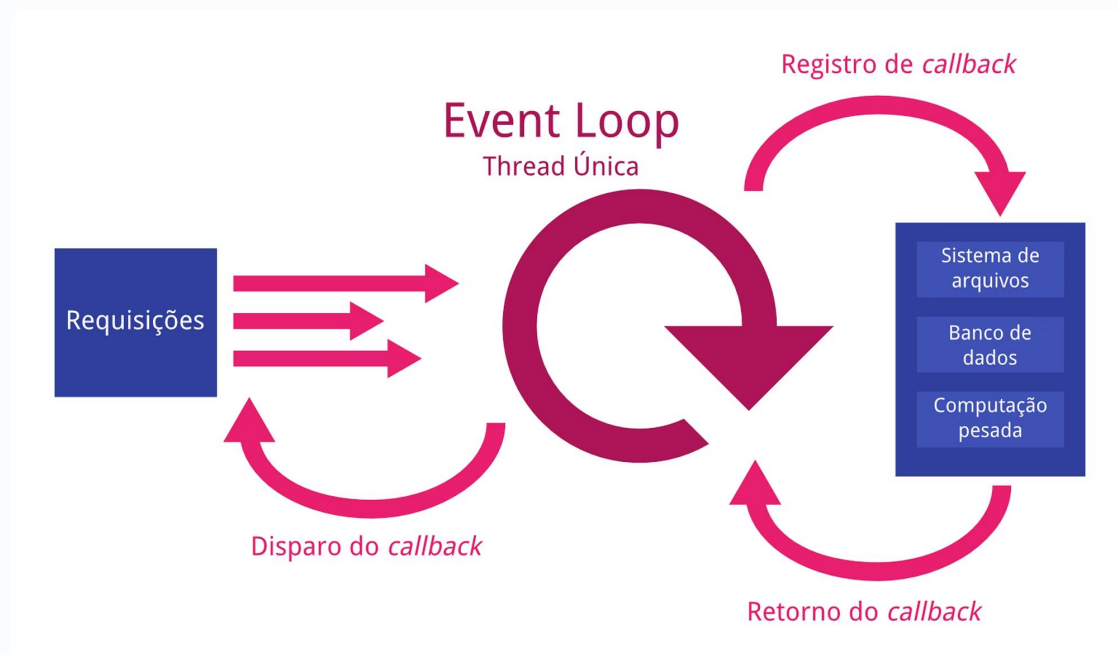
Você pode criar um servidor web com apenas algumas poucas linhas de código, além de contar com uma potente biblioteca de streams integrada.

NPM — Gerenciador de Pacotes

Um sistema gerenciador de pacotes amigável com milhares de módulos open-source disponíveis para escolha, acelerando o desenvolvimento de qualquer projeto.

Como o Node Funciona: o Event Loop

O Node trata conexões de forma diferente dos servidores tradicionais. Uma **única thread** recebe todas as conexões — ou seja, não há concorrência de recursos. Essa thread, chamada de **Event Loop** (laço de eventos), controla todos os fluxos assíncronos, eliminando o gargalo de um número máximo de requisições simultâneas.



No diagrama acima, as requisições chegam ao Event Loop, que registra *callbacks* para operações pesadas (sistema de arquivos, banco de dados, computação). Quando a operação termina, o *callback* é disparado e a resposta retorna ao cliente — sem bloquear o servidor.

Vantagens e Desvantagens do Node.js

✓ Vantagens

- **Usa JavaScript** — linguagem amplamente conhecida e com grande comunidade
- **JavaScript full-stack** — mesmo código no front end e no back end
- **Muito leve** — pouco investimento em infraestrutura e multiplataforma
- **Alta escalabilidade** — lida com milhares de conexões simultâneas com eficiência

⚠ Desvantagens

- **Tipagem fraca do JavaScript** — pode gerar bugs difíceis de rastrear em projetos grandes
- **POO fora do padrão** — orientação a objetos diferente de linguagens como Java ou C#
- **Tecnologia relativamente recente** — surgiu em 2009, ecossistema ainda em maturação
- **Uso excessivo de callbacks** — a natureza assíncrona pode dificultar a leitura do código
- **Não recomendado para cálculos pesados** — operações CPU-intensivas podem bloquear o Event Loop

Baixar o Node.js®

Baixe o Node.js® v24.18.0 LTS para Windows usando Docker com npm

Informação Quer novos recursos mais cedo? Obtenha a [versão mais recente do Node.js](#) e experimente as últimas melhorias!

```
1 # O Docker possui instruções específicas de instalação para cada sistema operacional.
2 # Consulte a documentação oficial em https://docker.com/get-started/
3
4 # Baixar a imagem Docker do Node.js:
5 docker pull node:24-alpine
6
7 # Criar um contêiner do Node.js e iniciar uma sessão Shell:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verifique a versão do Node.js:
11 node -v # Deve exibir "v24.18.0".
12
13 # Verificar a versão do npm:
14 npm -v # Deve imprimir "11.16.0".
```

PowerShell [Copiar para a área de transferência](#)

Docker é uma plataforma de containerização. Se você encontrar algum problema, por favor visite o [site do Docker](#).

Ou baixe uma versão pré-compilada do Node.js® para Windows rodando uma arquitetura x64.

[Instalador Windows \(.msi\)](#) [Binário Independente \(.zip\)](#)

Instalando o Node.js

Para começar a usar o Node.js, é necessário instalá-lo em seu computador. Siga os passos abaixo:

01

Acesse o site oficial

Entre em <https://nodejs.org/pt-br/download> e escolha a versão recomendada (LTS) para maior estabilidade.

02

Baixe o instalador

Selecione o arquivo correspondente ao seu sistema operacional (Windows, macOS ou Linux) e realize o download.

03

Execute a instalação

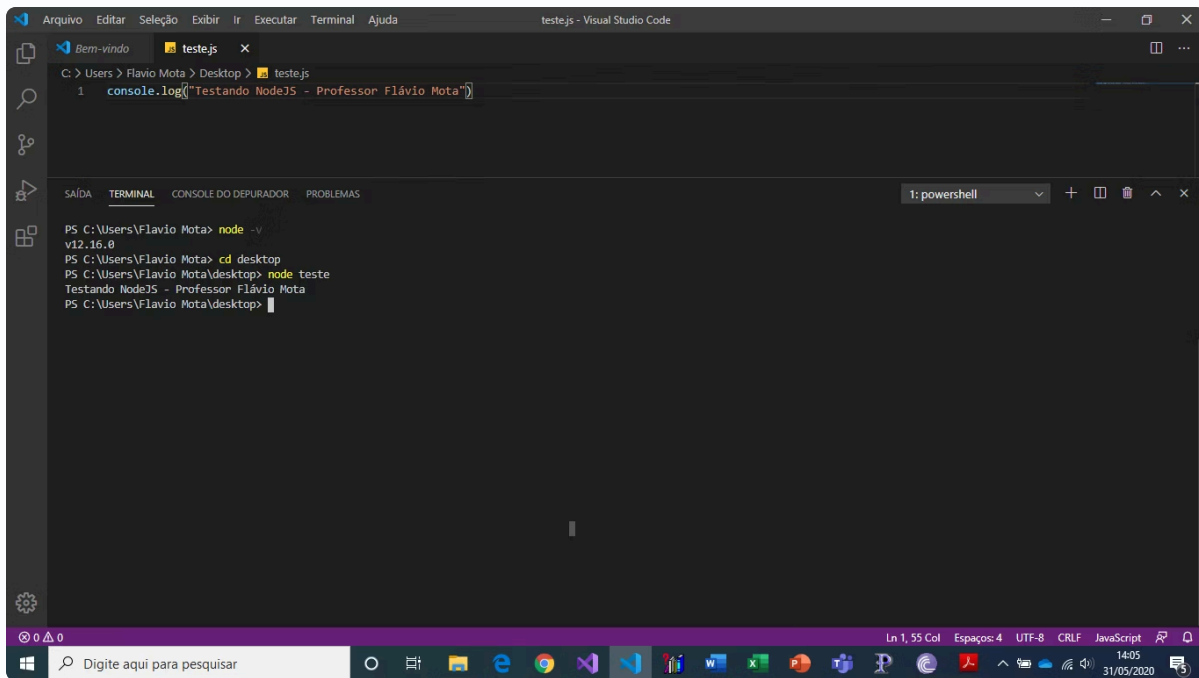
Abra o arquivo baixado e siga o assistente de instalação. Ao final, o Node.js e o NPM estarão disponíveis no terminal.

 Sempre instale a versão **LTS (Long Term Support)** para garantir maior estabilidade e suporte prolongado em seus projetos.

Executando Arquivos .js no Node

Para rodar qualquer arquivo .js no Node, abra o terminal de comandos do Windows ou o terminal do Visual Studio Code, navegue até a pasta onde está seu arquivo e execute o comando:

```
node arquivo.js
```



Arquivo teste.js no VS Code

Escreva seu código JavaScript normalmente no editor. O Node.js interpretará o arquivo do lado do servidor, sem necessidade de um navegador.

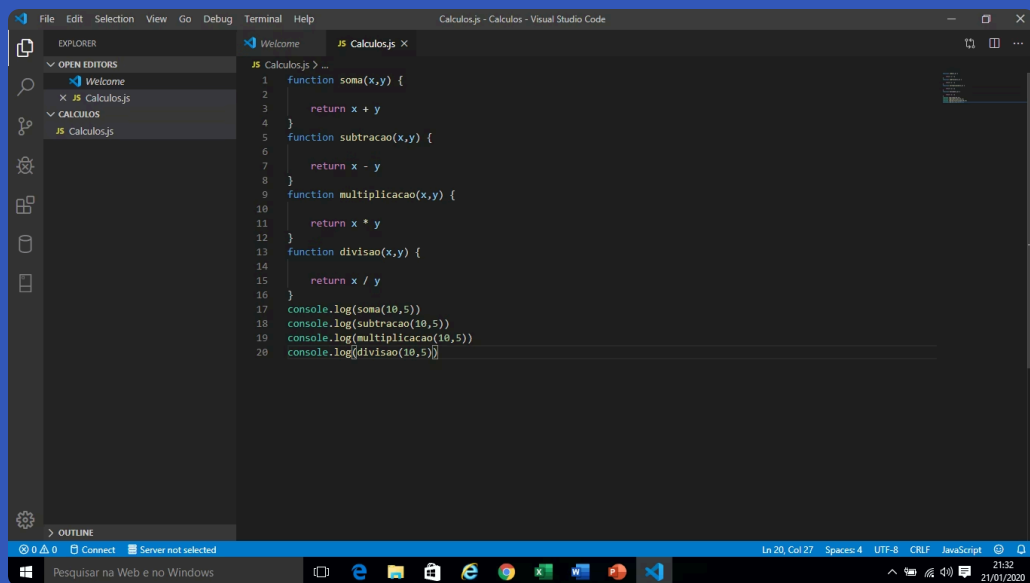
Resultado no terminal

Após executar o comando `node teste.js`, a saída do programa aparecerá diretamente no terminal do Visual Studio Code.

Executando JavaScript no Node: Exemplo Prático

Veja como um arquivo JavaScript com operações matemáticas é executado no Node.js. O exemplo abaixo demonstra a criação de um arquivo `calculos.js` e sua execução via terminal.

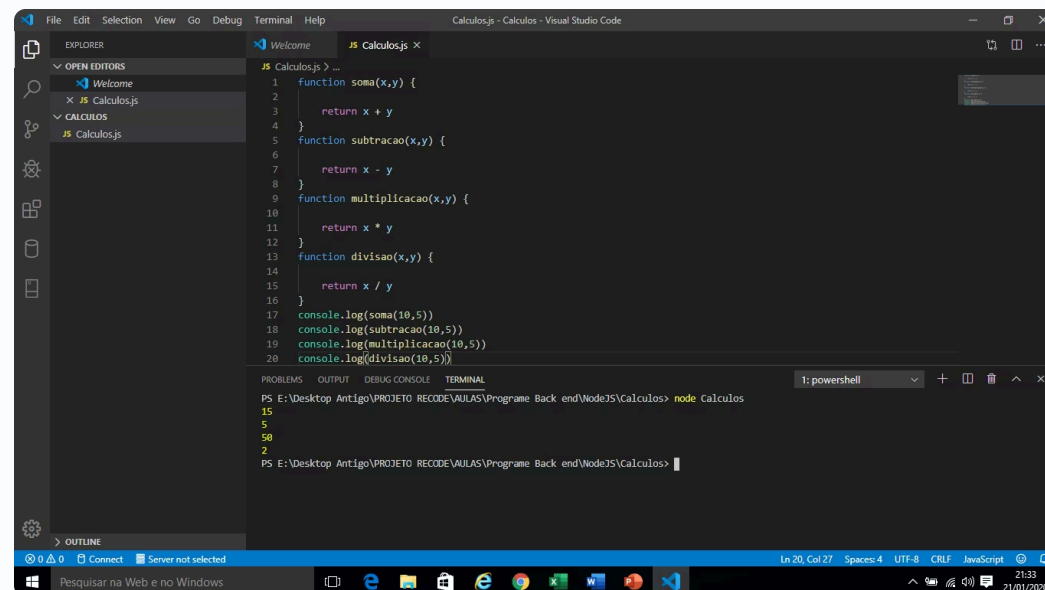
calculos.js

A screenshot of the Visual Studio Code editor with a dark theme. The Explorer sidebar on the left shows a file named 'calculos.js' under a folder named 'CALCULOS'. The main editor window displays the code for 'calculos.js'. The code defines four functions: 'soma' (addition), 'subtracao' (subtraction), 'multiplicacao' (multiplication), and 'divisao' (division). Each function takes two arguments, x and y, and returns the result of the operation. At the bottom of the file, there are four console.log statements, each calling one of the functions with the arguments 10 and 5.

```
1 function soma(x,y) {  
2     return x + y  
3 }  
4  
5 function subtracao(x,y) {  
6     return x - y  
7 }  
8  
9 function multiplicacao(x,y) {  
10     return x * y  
11 }  
12  
13 function divisao(x,y) {  
14     return x / y  
15 }  
16  
17 console.log(soma(10,5))  
18 console.log(subtracao(10,5))  
19 console.log(multiplicacao(10,5))  
20 console.log(divisao(10,5))
```

Arquivo com operações matemáticas básicas escritas em JavaScript puro.

Executando no Node

A screenshot of the Visual Studio Code editor, similar to the previous one, but with the Terminal panel open at the bottom. The terminal shows the command 'node calculos' being executed in a PowerShell prompt. The output of the script is displayed on the next line, showing the results of the four mathematical operations: 15, 5, 50, and 2.

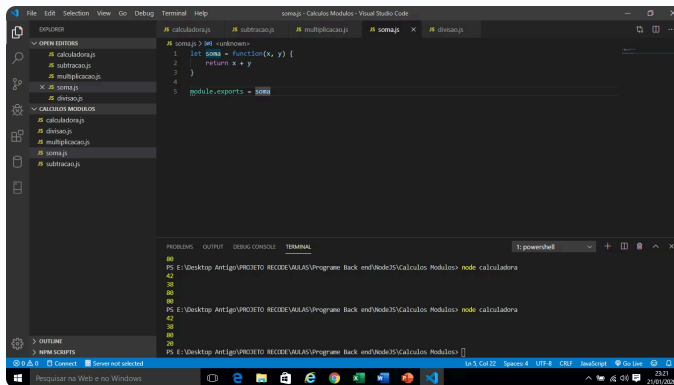
```
PS E:\Desktop Antigo\PROJETO RECODEVAULAS\Programa Back end\NodeJS\Calculos> node calculos  
15  
5  
50  
2  
PS E:\Desktop Antigo\PROJETO RECODEVAULAS\Programa Back end\NodeJS\Calculos>
```

Saída no terminal após executar `node calculos.js`. Os resultados das operações aparecem diretamente na linha de comando, confirmando que o Node interpretou e executou o arquivo com sucesso.

Módulos: exports e require()

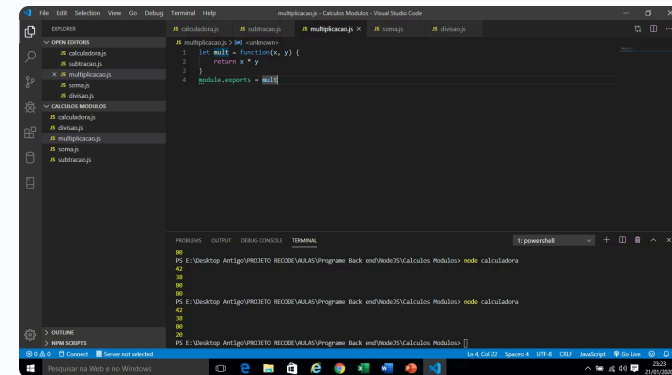
No Node.js, cada arquivo é tratado como um **módulo independente**. Para compartilhar funções entre arquivos, utilizamos `module.exports` para exportar e `require()` para importar. Abaixo, veja como criar quatro módulos matemáticos separados:

Módulo soma.js



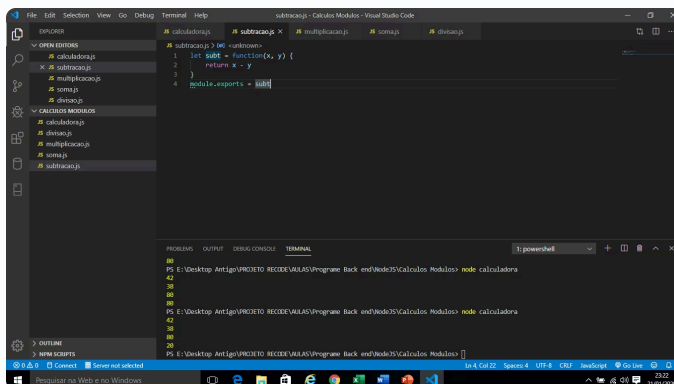
```
1 // soma.js
2
3 let soma = function(x, y) {
4   return x + y
5 }
6
7 module.exports = soma
```

Módulo multiplicação.js



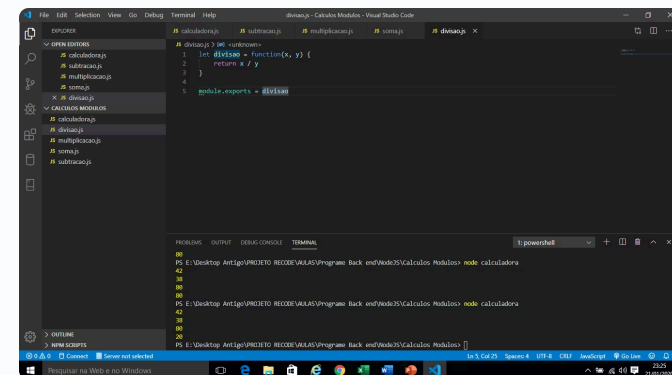
```
1 // multiplicacao.js
2
3 let mult = function(x, y) {
4   return x * y
5 }
6
7 module.exports = mult
```

Módulo subtração.js



```
1 // subtracao.js
2
3 let sub = function(x, y) {
4   return x - y
5 }
6
7 module.exports = sub
```

Módulo divisão.js



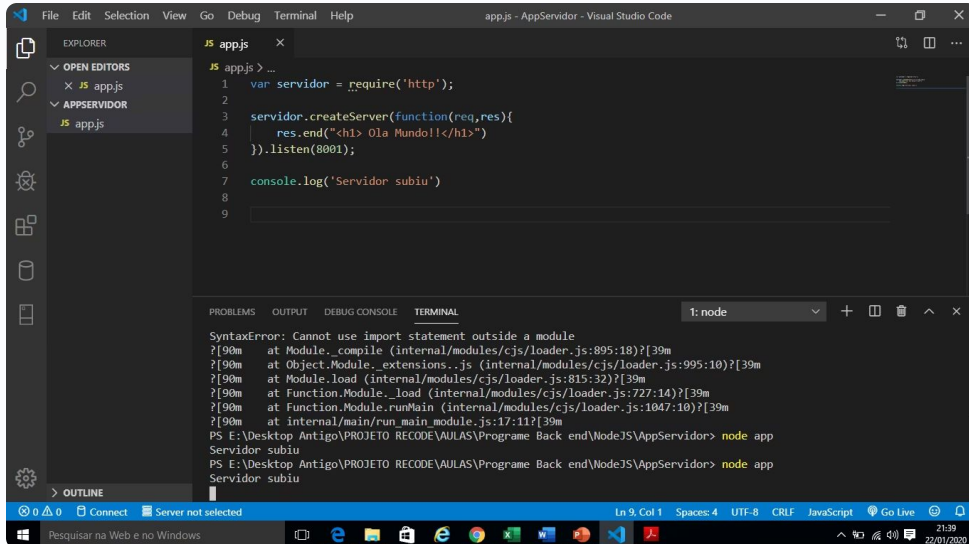
```
1 // divisao.js
2
3 let div = function(x, y) {
4   return x / y
5 }
6
7 module.exports = div
```

Módulos: Importando com `require()`

Módulos são cruciais para a construção de aplicações em Node, pois permitem incluir bibliotecas externas e organizar o código em partes separadas com responsabilidades bem definidas. A função

Criando Nosso Primeiro Servidor em Node

Com apenas algumas linhas de código, o Node.js permite criar um servidor web funcional usando o módulo nativo `http`. Veja como funciona o arquivo `servidor.js`:



```
1 var servidor = require('http');
2
3 servidor.createServer(function(req,res){
4   res.end("<h1> Ola Mundo!</h1>");
5 }).listen(8001);
6
7 console.log('Servidor subiu')
8
9
```

Problems Output DEBUG CONSOLE TERMINAL

1: node

SyntaxError: Cannot use import statement outside a module
[90m at Module._compile (internal/modules/cjs/loader.js:895:18)?[39m
[90m at Object.Module._extensions..js (internal/modules/cjs/loader.js:995:10)?[39m
[90m at Module.load (internal/modules/cjs/loader.js:815:32)?[39m
[90m at Function.Module._load (internal/modules/cjs/loader.js:727:14)?[39m
[90m at Function.Module.runMain (internal/modules/cjs/loader.js:1047:10)?[39m
[90m at internal/main/run_main_module.js:17:11)?[39m
PS E:\Desktop Antigo\PROJETO RECODE\AULAS\Programa Back end\NodeJS\AppServidor> node app
Servidor subiu
PS E:\Desktop Antigo\PROJETO RECODE\AULAS\Programa Back end\NodeJS\AppServidor> node app
Servidor subiu

require('http')

Importa o módulo nativo HTTP do Node e o atribui à variável `servidor`.

createServer()

Cria um novo servidor com uma função *callback* que recebe os parâmetros `req` (requisição) e `res` (resposta).

listen(8001)

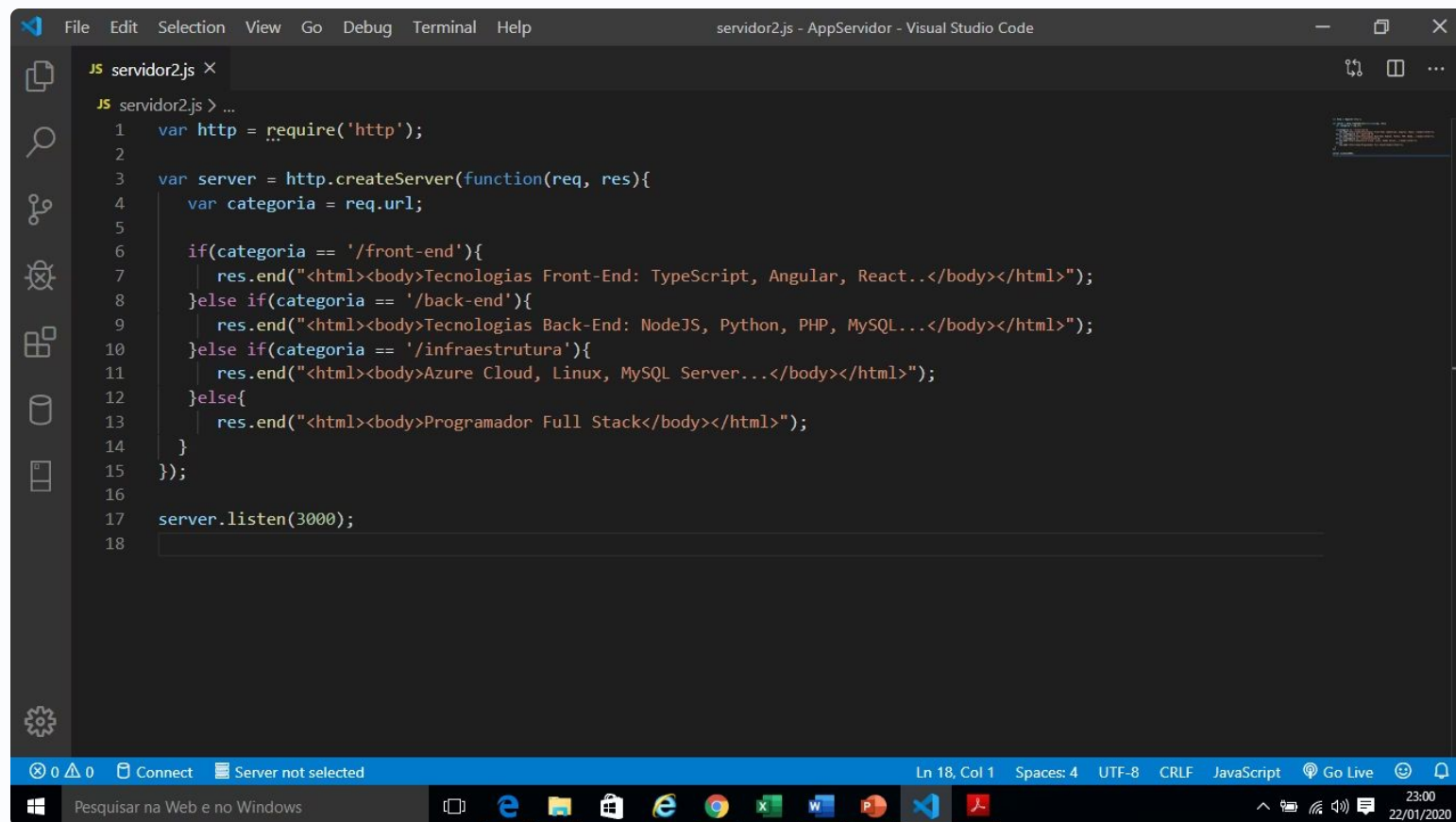
Define a porta em que o servidor ficará escutando conexões. Acesse via <http://localhost:8001/>

res.end()

Envia a resposta ao cliente e finaliza a conexão, exibindo o conteúdo no navegador.

Criando Rotas em Node — Parte 1

Rotas permitem que o servidor responda de forma diferente dependendo da URL acessada pelo usuário. No Node puro, utilizamos a propriedade `req.url` para identificar o caminho solicitado e direcionar a resposta correta. Veja a estrutura do arquivo `servidor2.js`:



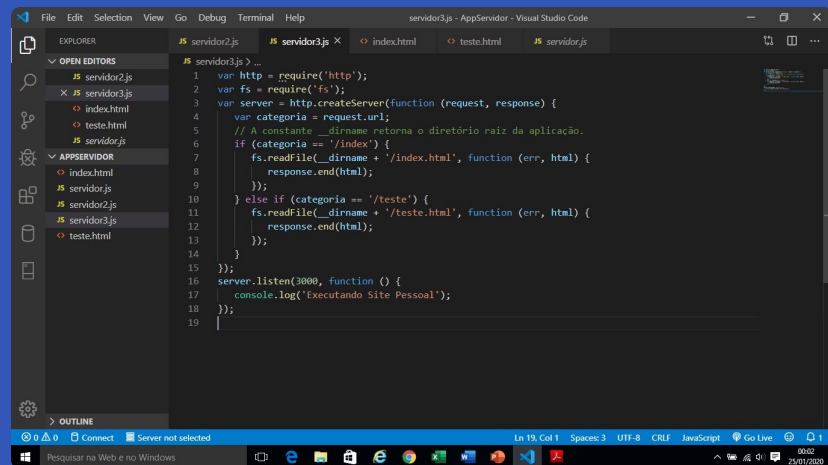
```
JS servidor2.js <
JS servidor2.js > ...
1  var http = require('http');
2
3  var server = http.createServer(function(req, res){
4      var categoria = req.url;
5
6      if(categoria == '/front-end'){
7          res.end("<html><body>Tecnologias Front-End: TypeScript, Angular, React...</body></html>");
8      }else if(categoria == '/back-end'){
9          res.end("<html><body>Tecnologias Back-End: NodeJS, Python, PHP, MySQL...</body></html>");
10     }else if(categoria == '/infraestrutura'){
11         res.end("<html><body>Azure Cloud, Linux, MySQL Server...</body></html>");
12     }else{
13         res.end("<html><body>Programador Full Stack</body></html>");
14     }
15 });
16
17 server.listen(3000);
18
```

Neste arquivo, verificamos o valor de `req.url` dentro do `callback` do servidor. Para cada rota identificada, enviamos uma resposta personalizada usando `res.end()`. Esse é o roteamento mais básico possível em Node.js puro, sem frameworks.

Criando Rotas em Node — Parte 2

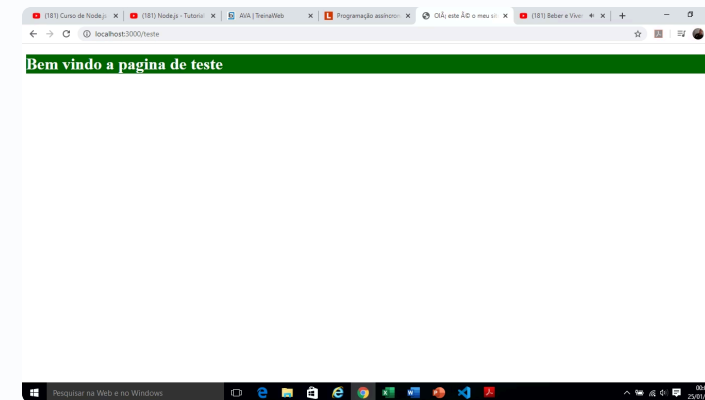
Além de responder com texto, é possível servir arquivos HTML para cada rota. O arquivo `servidor3.js` demonstra como ler e retornar arquivos HTML usando o módulo `fs` (filesystem) do Node.

Rota para index.html



```
1 var http = require('http');
2 var fs = require('fs');
3 var server = http.createServer(function (request, response) {
4   var categoria = request.url;
5   // A constante __dirname retorna o diretório raiz da aplicação.
6   if (categoria == '/') {
7     fs.readFile(__dirname + '/index.html', function (err, html) {
8       response.end(html);
9     });
10  } else if (categoria == '/teste') {
11    fs.readFile(__dirname + '/teste.html', function (err, html) {
12      response.end(html);
13    });
14  }
15 });
16 server.listen(3000, function () {
17   console.log('Executando Site Pessoal');
18 });
19 |
```

Rota para teste.html



Quando o usuário acessa `/teste`, o servidor lê o arquivo `teste.html` e o retorna como resposta HTTP. Cada rota pode ter seu próprio arquivo HTML correspondente, tornando o servidor capaz de servir páginas completas.



Trabalhando com o Framework Express

O **Express** é um framework para aplicações web do Node.js — mínimo, flexível e extremamente poderoso. Ele fornece um conjunto robusto de recursos para aplicações web e móveis, simplificando a criação de servidores e APIs.

01

Instale o Express no projeto

No terminal, dentro da pasta do projeto, execute:

```
npm install express --save
```

A cláusula `--save` registra o Express como dependência permanente do projeto no arquivo `package.json`. Sem ela, o pacote é instalado apenas para testes temporários.

02

Importe e instancie o Express

No arquivo principal, utilize `require('express')` e crie um objeto `app` que representa a instância do framework no seu projeto.

03

Configure rotas e inicie o servidor

Defina rotas com `app.get()`, `app.post()` etc., e use `app.listen(porta)` para colocar o servidor no ar.

Express: Criando o Primeiro Servidor

Com o Express instalado, crie um objeto `app` representando a instância do framework, defina uma rota inicial usando o método GET e configure a porta que o servidor irá escutar:

```
const express = require('express')
const app = express()

app.get("/", (req, res) => {
  res.send("Professor Flávio Mota")
})

app.listen(3000)
console.log('Servidor subiu')
```

Objeto app

Representa a instância do Express no projeto. Toda a configuração do servidor passa por ele.

Rota raiz /

O método `app.get('/')` define a resposta para a URL principal da aplicação.

Porta 3000

O servidor fica escutando na porta 3000. Caminhos não definidos retornam automaticamente **404 Não Encontrado**.

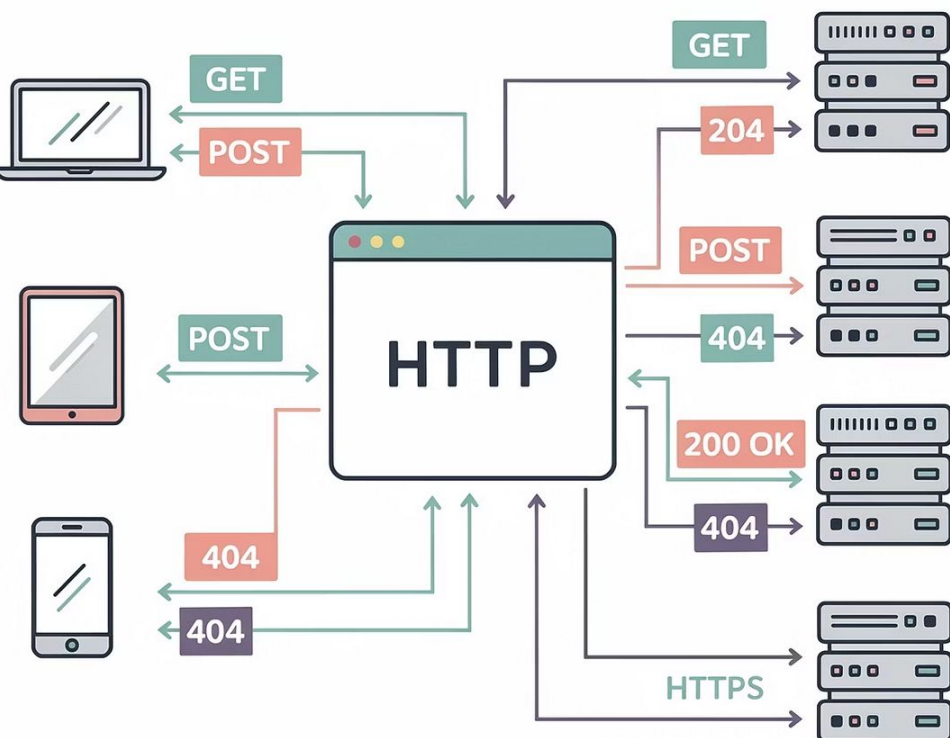
Roteamento no Express

O **roteamento** refere-se à determinação de como um aplicativo responde a uma solicitação do cliente por um endpoint específico — uma URI (caminho) e um método HTTP (GET, POST, etc.). Cada rota pode ter uma ou mais funções de manipulação executadas quando a rota é correspondida.

A definição de rotas aceita a seguinte estrutura:

Métodos HTTP no Express

O Express permite criar rotas com todos os métodos do protocolo HTTP, viabilizando a construção de **APIs REST** completas. Cada método tem uma semântica específica que define a intenção da requisição:



1

GET

Recupera dados do servidor. O servidor devolve informações ao cliente sem modificar recursos.

2

POST

Cria um novo recurso no servidor. Usado para enviar dados e incluir novas informações.

3

PUT

Atualiza um recurso existente no servidor. Substitui ou modifica os dados de um registro.

4

DELETE

Remove um recurso do servidor. Apaga permanentemente o registro identificado pela URL.

Método HTTP: GET

O método **GET** é utilizado para **recuperar dados** identificados pela URL. Requisições GET fazem com que o servidor devolva algo ao cliente — algo que está "dentro" do servidor. É o método padrão quando acessamos qualquer endereço no navegador.

```
app.get("/", (req, res) => {  
  res.send("Professor Flávio Mota")  
})
```

No exemplo acima, a rota `app.get('/usuarios')` retorna uma lista de usuários quando o cliente faz uma requisição GET para esse endereço. A resposta é enviada via `res.send()` ou `res.json()`.

 O GET nunca deve modificar dados no servidor — é apenas uma operação de leitura.

Método HTTP: POST

O método **POST** é utilizado para **criar um novo recurso** no servidor. Significa que estamos enviando dados para incluir uma nova informação — como cadastrar um usuário ou registrar um pedido.

```
app.post('/frontend/Salva', function(req, res) {  
  
    var dados = req.body;  
  
    connection.query('INSERT INTO conteudo SET ?', dados, function(error, result) {  
        res.redirect('/frontend');  
    });  
  
});
```

No exemplo, a rota `app.post('/usuarios')` recebe os dados do novo usuário via `req.body`, processa as informações e retorna uma confirmação. Para acessar o corpo da requisição, é necessário configurar o middleware `express.json()` na aplicação.

Método HTTP: PUT

O método **PUT** é utilizado para **atualizar um recurso existente** no servidor. Significa que queremos modificar os dados de um registro já cadastrado. Geralmente, o ID do recurso é passado como parâmetro na URL.

```
app.put('/conteudo/:id', (req, res) => {  
  const { index } = req.params; // recupera o index com os dados  
  const { name } = req.body;  
  
  dados[index] = name; // sobrepõe o index obtido na rota de acordo com o novo valor  
  
  return res.json(dados);  
}); // retorna novamente os dados atualizados após o update
```

No exemplo, `app.put('/usuarios/:id')` usa o parâmetro `:id` para identificar qual usuário será atualizado. O novo valor é recebido via `req.body` e o servidor retorna uma confirmação da atualização realizada.

Método HTTP: DELETE

O método **DELETE** é utilizado para **remover um recurso** do servidor. Significa que queremos apagar permanentemente um registro identificado pela URL. Assim como no PUT, o ID é geralmente passado como parâmetro.

```
app.delete('/conteudo/:id', (req, res) => {  
  const { index } = req.params; // recupera o index com os dados  
  
  geeks.splice(index, 1); // percorre o vetor até o index selecionado e deleta uma posição no array  
  
  return res.send();  
}); // retorna os dados após exclusão
```

No exemplo, `app.delete('/usuarios/:id')` localiza o usuário pelo ID informado na URL e o remove da base de dados. O servidor retorna uma confirmação de que o recurso foi excluído com sucesso.

⚠ Operações DELETE são irreversíveis. Sempre confirme a intenção do usuário antes de executar a exclusão em produção.

Roteamento Básico: Renderizando Arquivos HTML

Com o Express, é possível **renderizar arquivos HTML** diretamente como resposta a uma rota, utilizando o método `res.sendFile()`. A palavra reservada `__dirname` indica a pasta raiz da aplicação, garantindo que o caminho do arquivo seja sempre absoluto e correto, independentemente de onde o servidor seja executado.

```
const express = require('express')
const app = express()

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/src/artigos.html")
})

app.get("/artigos", (req, res) => {
  res.sendFile(__dirname + "/src/artigos.html")
})
```

```
app.get("/contato", (req, res) => {
  res.sendFile(__dirname + "/src/contato.html")
})

app.get("*", (req, res) => {
  res.status(404).sendFile(__dirname + "/src/erro.html")
})

app.listen(3333)
console.log('up')
```

O endereço `/src/artigos.html` indica o local do arquivo HTML dentro da estrutura do projeto. Dessa forma, cada rota pode servir uma página diferente, criando um site completo com Node.js e Express.

❏ Use `__dirname + '/src/artigos.html'` para garantir compatibilidade entre sistemas operacionais Windows, macOS e Linux.